

Faculty Of Engineering and Technology

Electrical and Computer Engineering Department

ENCS5341

Telco Customer Churn Prediction Using Machine Learning

Assignment 1

Prepared by:

Alaa Qalalwa, 1230189.

Instructor: Dr. Aziz Qaroush

Section: 1.

Date: 4|Sep|2026

Contents

Executive Summary 1

1. Dataset Description & Exploratory Data Analysis (Task 1) 1

2. Data Preprocessing & Train/Test Split (Tasks 2 & 3) 3

3. Feature Selection Analysis (Task 4) 3

4. Baseline Classifiers (Task 5) 4

5. Classification Models Benchmark (Task 6) 4

6. Hyperparameter Tuning (Task 7) 5

7. Class Imbalance Investigation (Task 8) 5

8. Cross-Validation & Metric Selection (Task 9) 6

9. ROC-AUC, Confusion Matrix & Business Cost Analysis (Task 10) 7

10. Data Leakage Challenge (Task 11) 9

11. Final Model Comparison (Task 12) 10

12. Deployment Recommendation (Task 13) 10

13. Required Final Analysis 11

List of Figures

1 Target variable (Churn) distribution. 2
2 Correlation heatmap of numerical features and the target. 3
3 Top-15 features by Mutual Information and Random Forest importance. 4
4 Recall, precision, and F1 across imbalance-handling strategies. 6
5 Combined ROC curves for the top-3 tuned models (out-of-fold CV). 7
6 Confusion matrices at the default vs. cost-optimal threshold. 8
7 Expected business cost as a function of decision threshold. 9
8 Cross-validation scores and generalization gap: correct vs. leaked pipeline. 10

List of Tables

1 Feature selection — cross-validated ROC-AUC by feature subset size. 4
2 Baseline classifier performance (Stratified 5-Fold CV). 4
3 Default classification models — Stratified 5-Fold CV performance. 5
4 Hyperparameter tuning results (RandomizedSearchCV, 5-Fold CV). 5
5 Class imbalance handling — cross-validated comparison and expected cost. 6
6 Confusion matrix summary and cost-optimal thresholds (top-3 models). 8
7 Data leakage experiment — correct vs. leaked pipeline (Random Forest). 9
8 Final model comparison — test-set performance and expected cost. 10

Executive Summary

Customer churn prediction is a cost-asymmetric problem: missing a customer who is about to leave is far more expensive than an unnecessary retention offer. This pipeline was built end-to-end with two hard constraints — leakage prevention (every preprocessing, resampling, and feature-selection step is fitted exclusively on training folds and never sees the held-out test set until the final evaluation) and cost-aware evaluation (the \$500 false-negative / \$50 false-positive structure drives every model-selection decision, not just raw accuracy). Five algorithms — Logistic Regression, SVM, Random Forest, XGBoost, LightGBM — were benchmarked with default hyperparameters, then tuned via `RandomizedSearchCV`, then combined with the best-performing class-imbalance strategy for each. A deliberate leakage experiment quantifies the exact danger the assignment warns against, rather than merely asserting it. The final recommendation is **Logistic Regression with balanced class weights, at a decision threshold of 0.18**, which achieves the lowest expected business cost of any pipeline tested (\$33,900 on the untouched test set).

Key findings: (1) the dataset is moderately imbalanced (26.5% churners) — a model predicting "no churn" for everyone already scores 73.5% accuracy while being worthless; (2) the full 30-feature set consistently outperforms reduced subsets in cross-validation, so no feature selection is applied in the final pipelines; (3) class weighting is the single most effective lever discovered, raising Logistic Regression's recall from 54% to 80% and cutting its expected cost by roughly 44%; (4) the intentionally leaked pipeline inflated CV ROC-AUC by 10.6 points while producing a model that scored *identically* to the correct one on real test data.

1. Dataset Description & Exploratory Data Analysis (Task 1)

The IBM Telco Customer Churn dataset contains 7,043 customers across 21 columns spanning demographics, account/contract details, subscribed services, and billing. `TotalCharges` is loaded as text rather than numeric because 11 records contain a whitespace string instead of a number — all 11 belong to customers with `tenure = 0` (brand-new accounts with no billing history yet), a subtle structural issue the assignment specifically warns about. These are coerced to `NaN` and later median-imputed.

Churn affects 1,869 of 7,043 customers (26.54%), a moderate imbalance that later proves severe enough to make accuracy a misleading metric (Section 4).

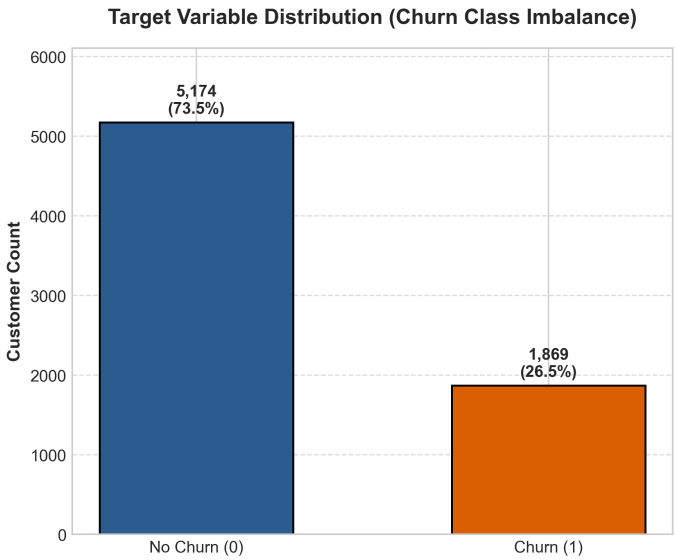


Figure 1: Target variable (Churn) distribution.

`tenure` (mean 32.4, skew 0.24) and `MonthlyCharges` (mean $64.76, skew -0.22) are both fairly symmetric; `TotalCharges` is right-skewed (skew 0.96) as expected for an accumulating quantity — long-tenured customers pull the distribution rightward. The IQR method (1.5×IQR beyond Q1/Q3) found **zero outliers** in any of the three numerical features, which is reassuring but also unsurprising, since none of these quantities has a natural mechanism to produce an extreme value in this dataset. Categorically, `Contract` skews toward month-to-month (55.0%), and this is one of the strongest churn signals in the data: churn rates are 42.7% (month-to-month) vs. 11.3% (one year) vs. 2.8% (two year) — customers without a long-term commitment have essentially no switching cost. `InternetService` shows a similar gradient (fiber optic 41.9% churn vs. DSL 19.0% vs. none 7.4%), plausibly reflecting price sensitivity or service-quality frustration among fiber subscribers, and `PaymentMethod` is dominated by electronic check (33.6% of customers), a segment that also churns disproportionately — commonly attributed to it being the default option for less-engaged, price-sensitive customers rather than a causal driver in itself.

The correlation matrix shows `tenure`–`TotalCharges` at r = 0.826 and `MonthlyCharges`–`TotalCharges` at r = 0.651 (expected, since `TotalCharges` is approximately `tenure` × `MonthlyCharges`), which motivates L2 regularization for the linear model. `tenure` has the strongest linear relationship with churn of any numeric feature (r = -0.352): the longer a customer stays, the less likely they are to leave.

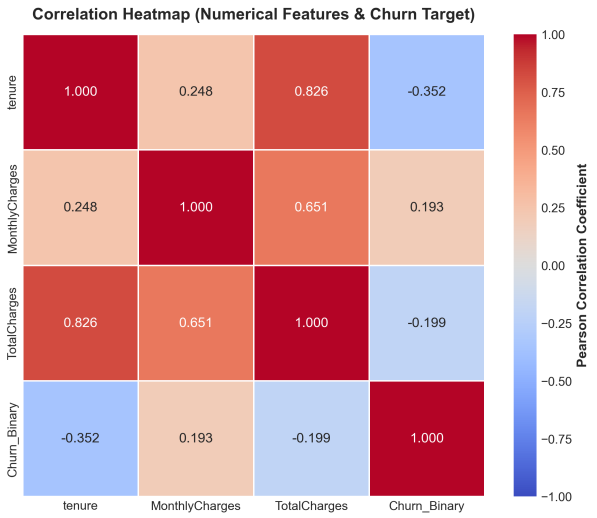


Figure 2: Correlation heatmap of numerical features and the target.

Modeling implications: the imbalance rules out accuracy as a primary metric (Section 8); the `TotalCharges` structural issue requires explicit numeric coercion before modeling; and the tenure–billing collinearity argues for regularized linear models and independent feature-selection validation (Section 3) rather than trusting a single ranking method.

2. Data Preprocessing & Train/Test Split (Tasks 2 & 3)

`customerID` is dropped (pure identifier). Numeric features (`tenure`, `MonthlyCharges`, `TotalCharges`) are median-imputed then standardized; median is used specifically because `TotalCharges` is right-skewed. The 16 categorical features are one-hot encoded with `drop='first'` (avoids the dummy-variable trap) and `handle_unknown='ignore'` (protects against unseen categories in a CV fold). All of this lives inside a single `ColumnTransformer`, itself the first step of an `imblearn.pipeline.Pipeline` — so every `cross_validate`/`RandomizedSearchCV` call refits it fresh on that fold's training partition only, never on the full dataset. This is the mechanism validated directly in Section 10.

The dataset was split 80/20 (5,634 train / 1,409 test) with stratification, **before** any feature selection, resampling, or tuning — churn rate is preserved at 26.54% in both sets. The test set is not touched again until Section 11.

3. Feature Selection Analysis (Task 4)

Two methods were run on the training set only: **Mutual Information** and **Random Forest importance**. Both agree on the core drivers — `tenure`, `Contract`, `MonthlyCharges`, `TotalCharges`, `PaymentMethod_Electronic check` — but diverge in ranking order (MI favors `Contract_Two year` at rank 2; RF importance ranks it 7th, favoring the three continuous billing features instead, since they offer more distinct split points).

Cross-validated performance was tested directly against three feature subsets:

Table 1: Feature selection — cross-validated ROC-AUC by feature subset size.

Feature Set	# Features	LR ROC-AUC	RF ROC-AUC
Top 10 (MI)	10	0.8338	0.7901
Top 15 (MI)	15	0.8379	0.7981
Full set	**30**	**0.8461**	**0.8241**

Performance rises monotonically with more features for both a linear and a tree model. **Decision: all final pipelines use the complete 30-feature set** — pruning measurably hurts rather than helps.

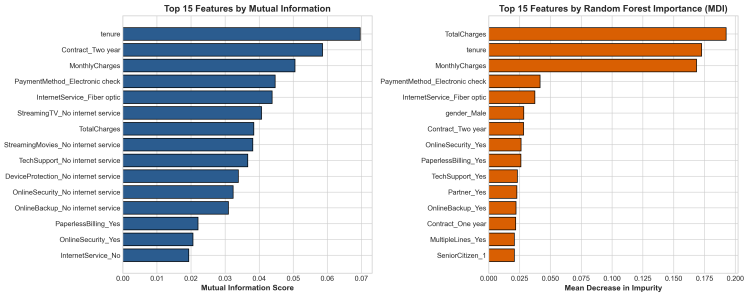


Figure 3: Top-15 features by Mutual Information and Random Forest importance.

4. Baseline Classifiers (Task 5)

Table 2: Baseline classifier performance (Stratified 5-Fold CV).

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Dummy (majority-class)	0.7346	0.0000	0.0000	0.0000	0.5000
Logistic Regression	0.8030	0.6555	0.5438	0.5938	0.8461

The DummyClassifier reaches 73.5% accuracy by always predicting "no churn" — a ROC-AUC of exactly 0.500 confirms zero discriminative power. This is the concrete proof that accuracy is the wrong primary metric here (Section 8): a model can look "80% accurate" while being barely better than a coin flip, unless it's measured against this floor.

5. Classification Models Benchmark (Task 6)

Logistic Regression gives directly interpretable, calibrated odds-ratio coefficients. **SVM (RBF)** trades interpretability for a non-linear margin-maximizing boundary, at the cost of an expensive Platt-scaling step for `probability=True`. **Random Forest** averages decorrelated trees, handling non-linearities without scaling. **XGBoost/LightGBM** build trees sequentially to correct residual error — XGBoost level-wise with heavy regularization, LightGBM leaf-wise for faster convergence at higher overfitting risk.

Table 3: Default classification models — Stratified 5-Fold CV performance.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8030	0.6555	0.5438	0.5938	**0.8461**
SVM (RBF)	**0.8042**	**0.6821**	0.4923	0.5717	0.7994
Random Forest	0.7872	0.6316	0.4763	0.5427	0.8241
XGBoost	0.7872	0.6186	0.5164	0.5627	0.8239
LightGBM	0.7936	0.6379	0.5137	0.5689	0.8365

SVM has the highest raw accuracy but the *lowest* ROC-AUC — an early sign that accuracy and discrimination can disagree (Section 9.4 makes this decisive).

6. Hyperparameter Tuning (Task 7)

`RandomizedSearchCV`, Stratified 5-Fold CV, 2–3 hyperparameters per model, optimizing ROC-AUC (SVM used 12 iterations on a subsample per the assignment's runtime note; others used 15).

Table 4: Hyperparameter tuning results (RandomizedSearchCV, 5-Fold CV).

Model	Best Parameters	Default AUC	Tuned AUC	Gain
Logistic Regression	C~12.0, solver=saga	0.8461	0.8463	+0.0002
SVM	rbf, gamma=0.01, C=1.0	0.7994	0.8367	**+0.0373**
Random Forest	n=150, depth=8, split=5	0.8241	0.8460	+0.0220
XGBoost	n=100, depth=3, lr=0.05	0.8239	**0.8488**	+0.0250
LightGBM	n=200, leaves=25, lr=0.01	0.8365	0.8460	+0.0095

SVM improved the most (its default gamma was a poor fit); RF and XGBoost both benefited from *shallower* trees, suggesting the untuned defaults were overfitting. Logistic Regression gained almost nothing — a well-regularized linear model has little headroom left to extract via search alone.

7. Class Imbalance Investigation (Task 8)

Three strategies were tested per model on top of tuned hyperparameters — **no intervention**, **class weighting** (`class_weight='balanced'` / `scale_pos_weight`), and **SMOTE** (inside `imblearn.Pipeline`, refit per fold) — converted into **Expected Cost** ($\$500 \cdot \text{FN} + \$50 \cdot \text{FP}$) from aggregate out-of-fold confusion counts. *(LR = Logistic Regression, RF = Random Forest, XGB = XGBoost, LGBM = LightGBM.)*

Table 5: Class imbalance handling — cross-validated comparison and expected cost.

Model	Strategy	Recall	Precision	F1	ROC-AUC	Exp. Cost
LR	None	0.5438	0.6555	0.5938	0.8461	$362,400
LR	**Weighted**	**0.8013**	0.5185	0.6294	0.8459	**$204,150**
LR	SMOTE	0.7906	0.5259	0.6314	0.8447	$209,800
RF	None	0.4763	0.6316	0.5427	0.8241	$412,300
RF	**Weighted**	0.6475	0.5812	0.6124	0.8282	**$298,350**
RF	SMOTE	0.5759	0.5887	0.5821	0.8217	$347,100
XGB	None	0.5164	0.6186	0.5627	0.8239	$385,300
XGB	**Weighted**	0.6542	0.5431	0.5934	0.8223	**$299,650**
XGB	SMOTE	0.5666	0.5881	0.5770	0.8214	$353,700
LGBM	None	0.5137	0.6379	0.5689	0.8365	$385,300
LGBM	**Weighted**	**0.7365**	0.5453	0.6266	0.8379	**$242,900**
LGBM	SMOTE	0.5893	0.5995	0.5941	0.8341	$336,550

Both interventions raise recall sharply (LR: 54%→80%) while cutting precision by 13–14 points — the textbook trade-off the assignment asks students to characterize. ROC-AUC barely moves under either intervention (typically less than half a point), which makes intuitive sense: ROC-AUC measures how well a model *ranks* churners above non-churners across every possible threshold, and neither weighting nor SMOTE changes that underlying ranking — they mainly shift *where* the model's default 0.5 boundary falls relative to it. **Class weighting wins for every model tested**, edging out SMOTE most clearly for Random Forest ($298,350 vs. $347,100); a plausible reason is that SMOTE's interpolations across mostly one-hot categorical dimensions produce less realistic synthetic points than simply reweighting the loss — interpolating "between" two dummy-encoded category vectors does not always correspond to a meaningful customer profile, whereas class weighting leaves the training data completely untouched. These four bold rows are exactly what Section 11 reads to select each model's final strategy — the choice is fully data-driven, not asserted by hand.

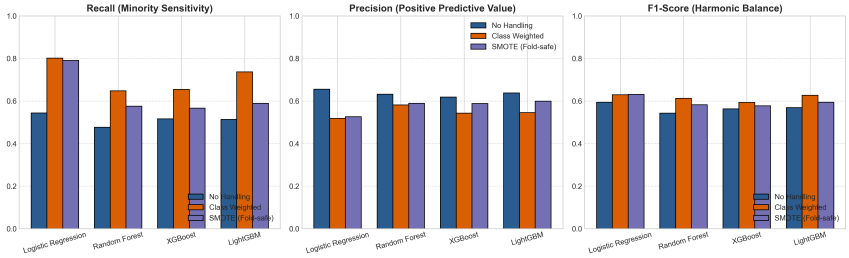


Figure 4: Recall, precision, and F1 across imbalance-handling strategies.

8. Cross-Validation & Metric Selection (Task 9)

Stratified 5-Fold CV was used throughout Sections 5–7, on the training set only; the test set is untouched until Section 11 — following *Training → CV → Model/Hyperparameter Selection → Final Model → Untouched Test Set.* Accuracy is the wrong primary metric here because a model that identifies zero churners still scores 73.5% (Section 4). **ROC-AUC** is the primary tuning/selection metric (threshold-independent); **Recall** and **Expected Cost** are the metrics that matter once a

specific threshold is chosen, since a missed churner costs ten times an unnecessary offer.

9. ROC-AUC, Confusion Matrix & Business Cost Analysis (Task 10)

The top 3 models by tuned CV ROC-AUC — **XGBoost** (0.8488), **Logistic Regression** (0.8463), **Random Forest** (0.8460) — were analyzed via out-of-fold predictions (training set), keeping the test set untouched.

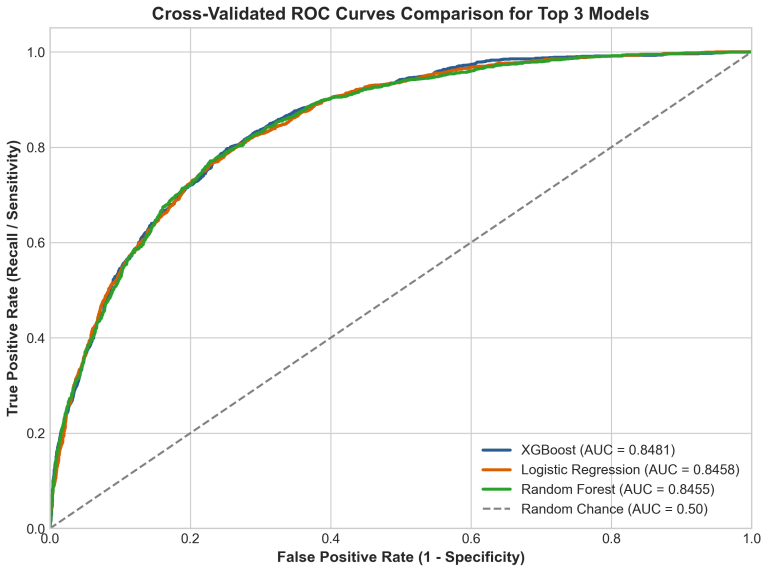


Figure 5: Combined ROC curves for the top-3 tuned models (out-of-fold CV).

All three curves are visually close (ROC-AUC within 0.003 of each other); differences only matter once a threshold is chosen. Confusion matrices at both the default and cost-optimal thresholds make the reason concrete:

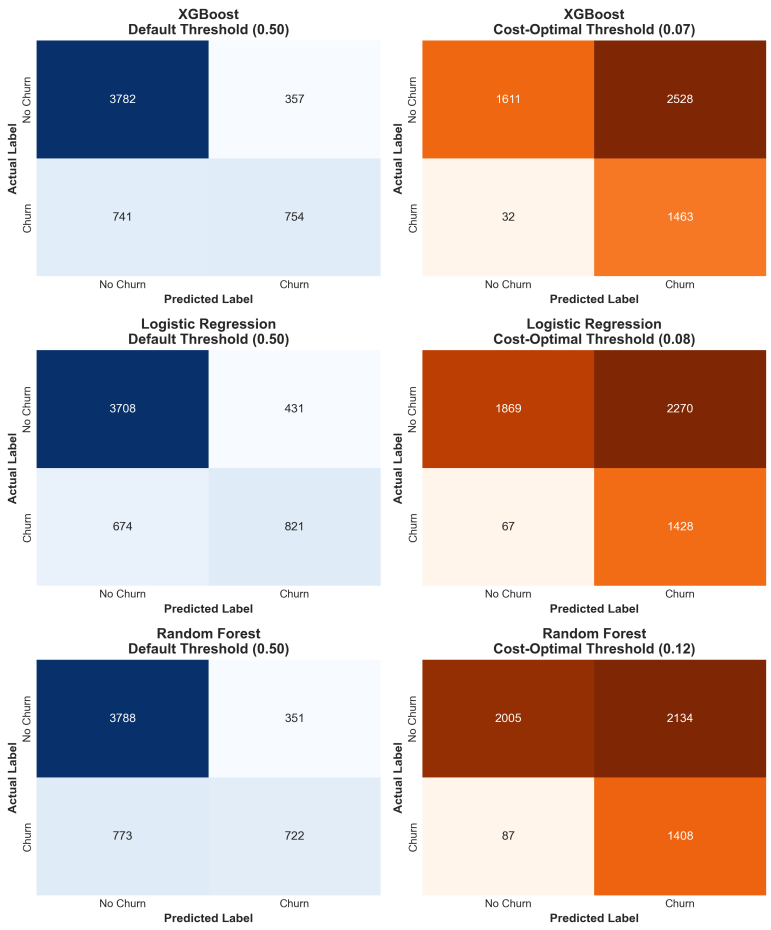


Figure 6: Confusion matrices at the default vs. cost-optimal threshold.

Table 6: Confusion matrix summary and cost-optimal thresholds (top-3 models).

Model	TN	FP	FN	TP	Recall	Cost (t=0.50)	Optimal t	Cost (optimal)
XGBoost	3,782	357	741	754	0.504	$388,350	0.07	$142,400
Logistic Regression	3,708	431	674	821	0.549	$358,550	0.08	$147,000
Random Forest	3,788	351	773	722	0.483	$404,050	0.12	$150,200

A **false negative** is a churner the model misses entirely — full future revenue lost; a **false positive** is an unneeded retention offer — a modest discount wasted, nothing more. At the default threshold all three models miss over 45% of churners, a direct consequence of training on a 26.5%-positive dataset without correcting for it: probability outputs are systematically compressed toward the majority class. Sweeping the threshold from 0.01 to 0.99 and minimizing expected cost at each point cuts cost by roughly 60% for every model:

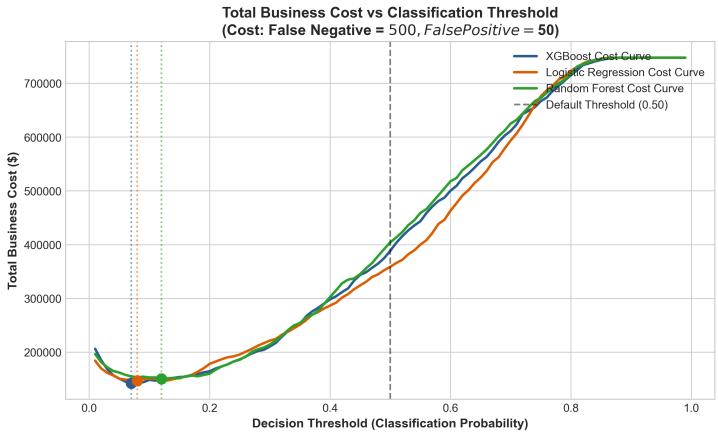


Figure 7: Expected business cost as a function of decision threshold.

These optimal thresholds (0.07–0.12) are much lower than Section 11's final thresholds (0.18–0.23) because Section 11's models already have class weighting applied, which partially recalibrates probabilities toward the minority class — it takes a smaller threshold shift to reach the same cost-minimizing point.

Business questions: (1) the 10:1 cost ratio means the model should tolerate many more false positives to minimize false negatives; (2) Recall and Expected Cost dominate over precision/accuracy; (3) the threshold should directly minimize 500·FN+50·FP, not default to 0.50; (4) **no** — SVM (Section 11) has the *highest* test accuracy (79.9%) of any final model yet the *worst* expected cost, because its recall is the lowest of the five.

10. Data Leakage Challenge (Task 11)

A Random Forest was compared under a **correct** pipeline (preprocessing + SMOTE inside `imblearn.Pipeline`, refit per fold) vs. a **leaked** one (SMOTE applied once to the full training set *before* CV, so synthetic points and their real neighbors can land in different folds).

Table 7: Data leakage experiment — correct vs. leaked pipeline (Random Forest).

	Correct	Leaked
CV Accuracy	0.7806	0.8600
CV Recall	0.5759	0.8649
CV ROC-AUC	0.8217	**0.9279**
Test ROC-AUC	0.8235	0.8235

The leaked pipeline looks 10.6 ROC-AUC points better in cross-validation — but scores **identically** to the correct pipeline on the real test set. This is, in a sense, the most convincing part of the experiment: it is not just that the leaked number is inflated, it is that the leaked training procedure produces a model that is **no better in reality** than the honest one, despite looking dramatically better on paper. The mechanism is straightforward: SMOTE's synthetic minority samples are generated as interpolations between a real point and its nearest same-class neighbors *before* the fold boundaries are drawn. When the resampled dataset is subsequently split into folds, a synthetic point and the real neighbor(s) it was derived from can land in different folds — so the model is effectively being partially tested on data that is a near-copy of what it just trained on.

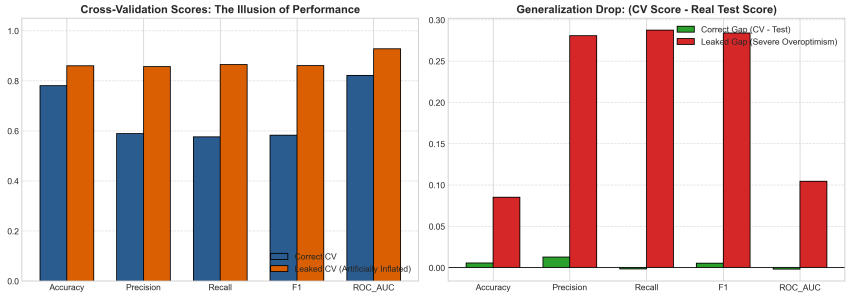


Figure 8: Cross-validation scores and generalization gap: correct vs. leaked pipeline.

Correct procedure: every data-dependent step (imputation, scaling, encoding, selection, resampling) must live inside a `Pipeline`/`imblearn.Pipeline` passed directly to `cross_validate`/`RandomizedSearchCV`, refit per fold. **This leaked run was never used for model selection or final reporting** — Sections 9 and 11 use only the leakage-safe pipelines from Sections 6–7.

11. Final Model Comparison (Task 12)

Each model's final pipeline combines its tuned hyperparameters (Section 6) with the imbalance strategy that minimized its expected cost in Section 7 — **class weighting for all four applicable models**. SVM received no imbalance handling (outside Task 8's scope). No feature selection is applied (Section 3). Models are ranked by **test-set expected cost at the optimal threshold**.

Model abbreviations: LR = Logistic Regression, RF = Random Forest, XGB = XGBoost, LGBM = LightGBM.

Table 8: Final model comparison — test-set performance and expected cost.

Model	Imb.	Test Acc.	Test Rec.	Test Prec.	Test F1	Test AUC	t*	Optimal Cost
LR	**Weighted**	0.7381	0.7834	0.5043	0.6136	0.8406	**0.18**	**$33,900**
XGB	Weighted	0.7410	**0.8102**	0.5075	0.6241	**0.8449**	0.19	$35,400
LGBM	Weighted	0.7644	0.7754	0.5390	**0.6360**	0.8442	0.22	$35,500
RF	Weighted	0.7530	0.7968	0.5228	0.6314	0.8439	0.23	$35,950
SVM	None	**0.7991**	0.4866	**0.6667**	0.5626	0.8298	0.11	$37,550

CV-to-test gaps are small everywhere (e.g., LR: 0.8462 CV AUC → 0.8406 test AUC), confirming the leakage-safe design generalizes — unlike Section 10's leaked experiment. Class weighting alone lifted LR's recall from 54% (Section 5) to 78%. **SVM is the clearest illustration of why accuracy is the wrong criterion**: highest accuracy and precision of the five, yet the highest expected cost, because of the lowest recall.

12. Deployment Recommendation (Task 13)

Tuned Logistic Regression with balanced class weighting, at threshold t* = 0.18, is recommended for deployment. It achieves the lowest expected cost of any pipeline tested ($33,900, roughly $1,500 below XGBoost and over $2,000 below Random Forest), the largest relative gain from threshold calibration of any model (a 38.3% cost reduction versus the default threshold), directly interpretable odds-ratio coefficients that a retention team or auditor can act on without

post-hoc explanation tools, and the cheapest inference cost of the five models (a single matrix multiplication and sigmoid, versus traversing an ensemble of 100–200 trees).

Why not XGBoost? It has the highest ROC-AUC (0.8449 vs. 0.8406) and recall (81.0% vs. 78.3%) of the two, and is a completely defensible second choice. But under the assignment's cost structure, that extra recall does not translate into a lower dollar cost — XGBoost's higher false-positive count at its own optimal threshold outweighs the gain, leaving it $1,500 more expensive with no interpretability advantage. It remains a reasonable challenger model or ensemble partner.

Why not Random Forest or LightGBM? Both post higher expected cost ($35,950 / $35,500) than Logistic Regression and XGBoost, with no compensating advantage in interpretability, and meaningfully higher inference cost than a linear model.

Why not SVM? It has the highest test accuracy (79.9%) and precision (66.7%) of any final pipeline, yet the lowest recall by a wide margin (48.7%) drives the *highest* expected cost of all five — nearly triple Logistic Regression's cost at the default threshold. It also received no imbalance-handling intervention within this project's scope and is, independently, the most expensive model to train and score, owing to the internal probability-calibration overhead that `SVC(probability=True)` requires.

13. Required Final Analysis

1. **Best overall performance:** by expected cost, Logistic Regression (Weighted); by ROC-AUC alone, XGBoost.
2. **Best recall:** XGBoost (81.0% test recall).
3. **Best ROC-AUC:** XGBoost (0.8488 CV / 0.8449 test).
4. **Imbalance handling:** raised recall 15–24 points across all models while cutting precision comparably; ROC-AUC barely moved (Section 7).
5. **Tuning impact:** meaningful for SVM/RF/XGBoost (+2 to +4 AUC points via depth/kernel fixes); negligible for Logistic Regression.
6. **Feature selection:** did not help — performance rose monotonically with more features (Section 3); the full 30-feature set is used.
7. **Leakage evidence:** leaked SMOTE inflated CV ROC-AUC by 10.6 points while test performance stayed identical to the correct pipeline (Section 10).
8. **Best metric:** ROC-AUC for model selection; Expected Cost ($\$500 \cdot \text{FN} + \$50 \cdot \text{FP}$) for the deployment decision.
9. **Cost effect on selection:** flips the winner from XGBoost (by ROC-AUC) to Logistic Regression (by cost), and justifies operating every model well below the default 0.50 threshold.
10. **Deployment choice:** Tuned, weighted Logistic Regression at $t^*=0.18$ — lowest cost, most interpretable, cheapest to run.

Figures: `outputs/figures/`. Result tables: `outputs/tables/`. `random_state = 42` fixed throughout for reproducibility.